StanfordQuadruped 第 4 讲讲义：MuJoCo 闭环仿真、状态回灌与调参

这份讲义对应系列课程的第 4 讲，目标是把 MuJoCo
仿真讲成“共享控制核心的闭环实验场”，而不是孤立的演示脚本。

建议时长：40~50 分钟

主文件：

- `sim/run_fixed_base.py`
- `sim/run_floating_base.py`
- `sim/sim_robot.py`
- `sim/task_scheduler.py`

本讲目标

- 讲清楚固定机身版和浮动机身版分别在学什么
- 看懂 `sim/sim_robot.py` 是怎样把 MuJoCo 包装成“像机器人一样”的接口
- 讲清楚状态回灌、PD 力矩驱动和任务调度的意义
- 给出一套真实可用的调参顺序

为什么第 4 讲才进入仿真细节

因为前 3 讲已经把共享控制核心讲完了。到了这一步再讲 MuJoCo，观众更容易意识到：

- 仿真不是另一套控制器
- 仿真是“控制器核心 + 仿真后端 + 观测桥接”的组合

`run_fixed_base.py` 和 `run_floating_base.py` 的分工

固定机身版

- 把控制器输出的关节角直接写入 `qpos`
- 不引入关节力矩、机身动力学和状态回灌
- 最适合检查 IK、足端轨迹和姿态命令

浮动机身版

- 机身使用 `freejoint`
- 关节由 PD 力矩驱动
- MuJoCo 里的姿态、速度、关节状态和触地回写到 `State`
- 最适合讲真正的闭环和调参

浮动机身控制流：任务层、控制器、执行器、MuJoCo、状态回灌

`sim/sim_robot.py` 是本讲的灵魂文件

这一个文件里其实放了五个桥接模块：

- `SimObservationInterface`
- 把 MuJoCo 传感器写回 `State`
- `SimIMU`
- 让仿真也长得像实机 IMU
- `SimHardwareInterface`
- 把关节目标变成 PD 力矩
- `SimControlClock`
- 按控制周期推进多个仿真子步
- `TaskCommandSource`
- 不依赖 UDP 手柄，直接在本地生成 `Command`

`sync_state()` 为什么很值得讲

观众通常第一次真正理解“闭环”，就是在这里。

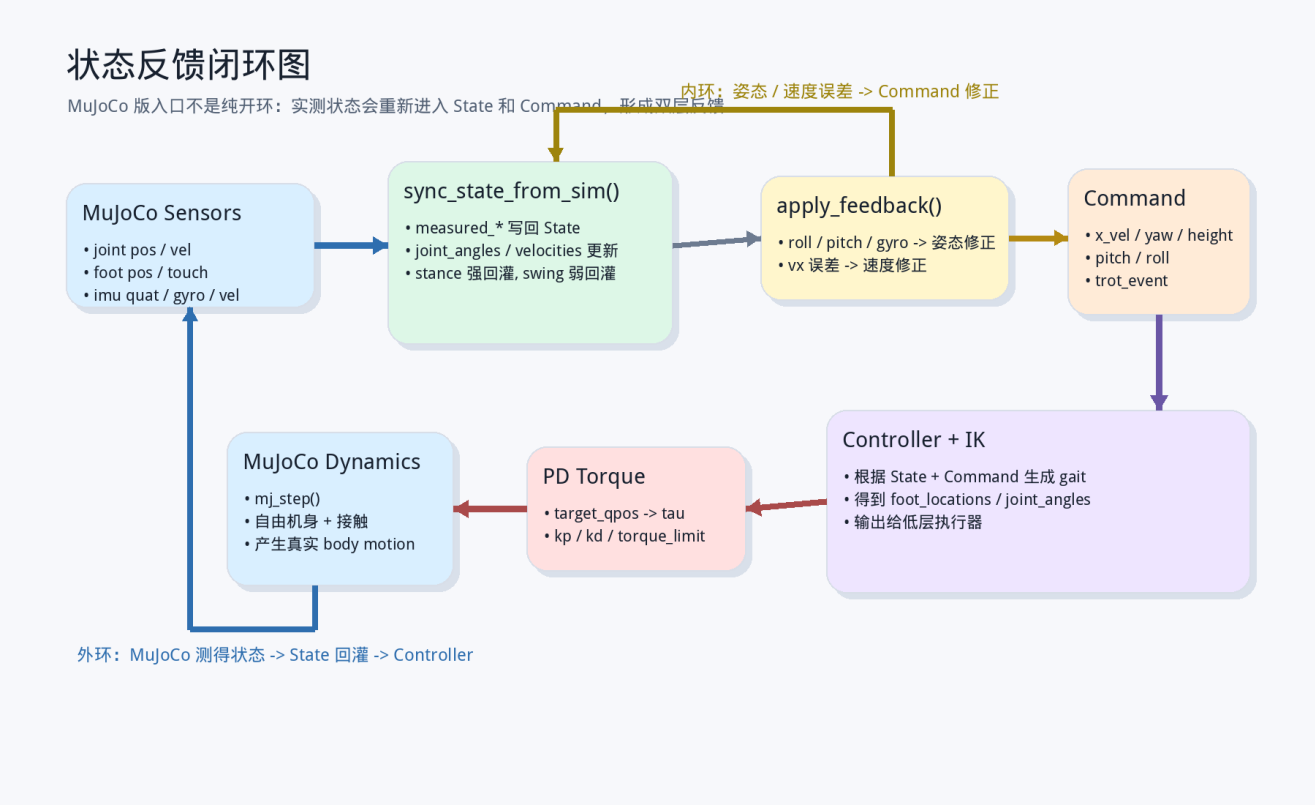
`sync_state()` 做了这些事：

- 读机身位置、姿态、速度、角速度
- 读足端位置和触地传感器
- 读关节角和关节角速度
- 更新 `state.measured_*`

- 把部分测量值融合回 `state.foot_locations`

这里一定要强调：

- 这个项目不是“纯开环轨迹播放”
- 它在仿真里已经有了明确的状态回灌



闭环状态反馈：姿态、速度、接触状态会反向影响控制器

`SimHardwareInterface` 怎么讲

它的意义很简单：

- 控制器仍然输出的是关节目标角
- 仿真执行器不是舵机 PWM，而是 MuJoCo motor
- 所以需要一层 `target_qpos -> PD torque` 的桥接

这层桥接刚好也能顺手讲两个控制直觉：

- `kp` 太小，跟踪软
- `kp` 太大、`kd` 不够时，系统容易振荡

`TaskScheduler` 和 `TaskCommandSource` 很适合录视频

这是当前仓库里最“教学友好”的新能力。

它解决的问题是：

- 没有手柄时，怎样安排一串可重复的实验
- 怎样把 `rest -> trot -> rest` 做成固定流程
- 怎样在不同时间片里覆盖 `vx / height / z_clearance / overlap_time`

任务语法可以直接给观众：

```
mode[:duration][@key=value;key=value...],mode[:duration],...
```

比如：

```
python sim/run_floating_base.py --duration 8 \
  --task-sequence "rest:1.0,trot:4.0@vx=0.08;z_clearance=0.04,rest" \
  --no-plots
```

本讲最推荐的调参顺序

1. 先固定 `rest`，确认姿态和初始高度正常
2. 再用 `fixed-base` 验证足端轨迹和 IK
3. 再切到 `floating-base`，先小速度前进
4. 再调 `kp / kd / torque-limit`
5. 再调 `overlap_time / swing_time / z_clearance`
6. 最后再引入 `attitude_kp / attitude_kd / velocity_kp`

最适合在视频里展示的症状与解释

- 机身前倾过大
- 看 `pitch`、`attitude_kp` 和 `velocity_kp`
- 腿拖地
- 看 `z_clearance`
- 前进发虚或不肯走
- 看 `kp`、`torque-limit` 和 `vx`
- 容易摔
- 看 `overlap_time` 是否太小、速度是否太大

本讲建议演示

```
python sim/run_floating_base.py --duration 8 --mode rest --no-plots
python sim/run_floating_base.py --duration 8 --mode trot --settle 1.0
python sim/run_floating_base.py --duration 8 \
  --task-sequence "rest:1.0,trot:4.0@vx=0.08;attitude_kp=0.03;velocity_kp=0.2,rest"
```

本讲作业

- 设计一条三段任务序列并解释每段目的
- 让观众自己回答：为什么 fixed-base 和 floating-base 要分开学
- 让观众写出一套最小调参顺序